

Plan de Integración: Memoria Virtual con Paginación y TLB (PatatOS v2)

Este documento detalla el plan para integrar el simulador de memoria virtual con paginación en el proyecto actual (Proy_SO_2026_1). El sistema mantendrá la compatibilidad total con la implementación actual, añadiendo la paginación como una opción adicional configurable desde la interfaz.

La arquitectura simulada representará un sistema de 32 bits, lo cual define la estructura de las direcciones virtuales. Todo el desarrollo se realizará de forma aislada utilizando control de versiones.

0. Estrategia de Control de Versiones

Para evitar cualquier posible ruptura de las funcionalidades actuales en producción o en la rama principal (main / master), todo el código de esta implementación se desarrollará y probará en una nueva rama de Git independiente.

- Nombre de la rama: feature/virtual-memory
- Proceso: Se creará esta rama a partir de la rama principal actual. Una vez que la implementación del Frontend y el Backend estén probadas y verificadas de extremo a extremo sin conflictos, se realizará la fusión (merge) de regreso a la rama principal.

1. Diseño y Arquitectura de Memoria Virtual

La arquitectura se extenderá para soportar memoria virtual basándose en las siguientes estructuras:

1. Direcciones de 32 bits y Tamaño de Página (4KB):
 - Se utilizará un tamaño de página fijo y estándar de 4 KB (bytes).
 - Una dirección virtual de 32 bits se dividirá en:
 - Offset: 12 bits (para direccionar los 4096 bytes dentro de la página).
 - VPN (Virtual Page Number): 20 bits (permitiendo hasta páginas).
2. Marcos de Página (Frames): La memoria física RAM será dividida en bloques de tamaño fijo de 4 KB.
3. Tablas de Páginas (Page Tables): Estructuras para la traducción de Memoria Lógica a Física (VPN a PFN). Soportará:
 - Nivel Simple (Single-Level).
 - Dos Niveles (Two-Level).
 - Tabla Inversa (Inverted).
 - Tabla Hash (Hashed).
4. TLB (Translation Lookaside Buffer): Se implementará una caché de traducción (TLB) simulada en la MMU con número de entradas configurable.
5. Gestor Swap (Swap Manager): Controlador de almacenamiento secundario simulado que tendrá un tamaño total configurable en MB y soportará diferentes penalizaciones de latencia según el tipo (HDD o SSD).

Algoritmos de Reemplazo de Páginas

Cuando la memoria RAM (frames) se llena, se deberá seleccionar una página víctima. El sistema implementará la siguiente lista de algoritmos seleccionables:

1. NRU (Not Recently Used): Clasifica las páginas en 4 clases (0 a 3) combinando los bits R (Referenciada) y M (Modificada).
2. FIFO (First-In, First-Out): Elimina la página que lleva más tiempo en memoria utilizando una cola.
3. Segunda Oportunidad: Variante de FIFO que perdona a la página si su bit R=1.
4. Reloj (Clock): Optimización circular de la Segunda Oportunidad.
5. LRU (Least Recently Used): Reemplazo de la página que lleva más tiempo sin ser referenciada.
6. NFU (Not Frequently Used): Aproximación de LRU que usa un contador alimentado por el bit R.
7. Envejecimiento (Aging): Mejora sobre NFU desplazando bits e insertando R en la posición más significativa.
8. Working Set (Conjunto de Trabajo): Mantiene en memoria el conjunto de páginas usadas activamente.
9. WSClock: Combina Reloj con Working Set.

2. Modificaciones en el Backend (C++)

En versiones previas, la MMU existía pero no era funcional. Con esta actualización, la MMU orquestará la traducción de direcciones y validación de fallos de página.

Tareas Backend:

1. Activación y Reparación de la MMU:
 - Interceptará todas las lecturas/escrituras. Consultará primero la TLB.
 - Si hay fallo en TLB, buscará en la PageTable. Si la página es inválida, levanta el Page Fault.
2. Tipos y Configuración (types.hpp, json_reader.cpp):
 - Agregar configuración de tipo de tabla, TLB, dispositivo swap, algoritmo y el tamaño máximo del Swap.
3. Refactor del Gestor de Memoria (PagedMemoryManager):
 - Interactuará con FrameTable, SwapManager y la PageTable. Actualizará bits R y M.
 - El SwapManager controlará que no se exceda el tamaño configurado de Swap; si se llena, se deberá aplicar una política de rechazo de memoria (Out of Memory).
4. Manejo de Fallos (Page Faults) en simulator.cpp:
 - La instrucción se pausa, enviando el proceso a BLOCKED_PAGEFAULT. Al transcurrir la latencia HDD/SSD, carga la página del Swap.
5. Exportación (json_writer.cpp):
 - Reportar estado de la TLB, fallos de página, los marcos de memoria física, y la capacidad/uso total del Almacenamiento Secundario (Swap).

3. Modificaciones en el Frontend (Python / PySide6)

El Frontend configurará y visualizará la memoria paginada, la TLB, y el almacenamiento secundario.

Tareas Frontend:

1. Configuración (ui/config_dialog.py):
 - Activar Modo Paginado (4KB / 32 bits).
 - Mostrar el combobox con los 9 algoritmos de reemplazo.
 - Añadir un selector numérico para definir el "Tamaño de Swap (MB)", permitiendo al usuario decidir cuánto espacio de almacenamiento secundario estará disponible.
2. Visualización Principal del Memory Map (ui/main_window.py / ui/memory_view.py):
 - RAM: Mostrar una cuadrícula principal de Frames físicos de 4KB.
 - Almacenamiento Secundario (Swap): En el mismo widget del Memory Map, justo debajo o al lado de la cuadrícula de RAM, se añadirá un indicador visual del Swap (una barra de progreso o mini-cuadrícula) escalada respecto al tamaño configurado.
3. Subventana Flotante no Intrusiva:
 - En el componente del Memory Map, se añadirá un ícono no intrusivo (ej. una pequeña lupa en una esquina).
 - Al hacer clic, aparecerá un QDialog flotante mostrando:
 - Entradas de la TLB.
 - La Tabla de Páginas del proceso seleccionado (mostrando bits V, R, M).
 - Detalle numérico del uso del Swap (páginas almacenadas por proceso).

>> SUGERENCIA

Ejecución

El diseño se ha perfeccionado integrando el control de tamaño sobre el almacenamiento Swap desde la UI, y garantizando su correcta gestión y visualización. El documento final ha sido pulido para tu aprobación.